

Nano-DDP

CSIT 5970 Project Report

Wu Yongjin 20564741

1. Introduction

The rapid advancement of large language models (LLMs) has dramatically increased the computational demands of model training. Early neural language models were typically trained on a single GPU or a small number of devices, but modern foundation models such as OpenAI GPT series, Google Gemini, and Meta Llama are trained using thousands of GPUs over extended periods. As model size grows from millions to hundreds of billions of parameters, the memory footprint and computational cost exceed the capacity of any single accelerator.

To address the enormous computational demands of modern large language models, distributed training has become an essential component of large-scale deep learning systems. By partitioning workloads across multiple GPUs, distributed training enables practitioners to scale model size, batch size, and training throughput beyond the limits of a single device. Depending on which components (data, weight) are distributed and how they are partitioned, several mainstream parallelization strategies have emerged, including Data Parallelism (DP), Pipeline Parallelism (PP), Tensor Parallelism (TP), Sequence Parallelism (SP), and Expert Parallelism (EP). These techniques are often combined in practice to form highly scalable training systems capable of supporting today's state-of-the-art LLMs.

Among these techniques, data parallelism (DP) remains the conceptual foundation of distributed training and is often the first strategy implemented in practice. PyTorch [1] Distributed Data Parallel (DDP) is the standard DP implementation in the PyTorch ecosystem, providing efficient gradient synchronization through communication primitives such as all-reduce and optimization techniques such as overlap between communication and computation.

In this project, I implement a simplified Nano-DDP system using only PyTorch's collective communication primitives and *autograd* hook APIs, which reproduces the core functionality of DDP. I then conduct a comprehensive performance evaluation, including scalability

experiments across different numbers of GPUs, comparisons against the official PyTorch DDP implementation as a baseline, and NVIDIA Nsight Systems profiling to analyze the overlap between computation and communication. Through this project, I aim to bridge the gap between the conceptual understanding and practical implementation of distributed LLM training systems.

2. Preliminary Knowledge

2.1 Data Parallelism

In the case of single GPU, the gradient for a mini-batch $\mathcal{B}$ of size B is calculated as:

$$g = \nabla_{\theta} \left(\frac{1}{B} \sum_{(x,y) \in \mathcal{B}} L(f(x; \theta), y) \right)$$

The optimizer then updates the model parameters using this gradient.

To scale up to multiple GPUs by Data Parallelism (DP), the global mini-batch $\mathcal{B}$ is partitioned across N GPUs into local mini-batches $\mathcal{B}_1, \dots, \mathcal{B}_N$. Each GPU computes a local gradient:

$$g_k = \nabla_{\theta} \left(\frac{1}{|\mathcal{B}_k|} \sum_{(x,y) \in \mathcal{B}_k} L(f(x; \theta), y) \right)$$

The final gradient used for the update is obtained by averaging the local gradients:

$$g = \frac{1}{N} \sum_{k=1}^N g_k$$

This is mathematically equivalent to computing the gradient over the full global batch on a single GPU.

A major optimization for the above naïve DP is overlapping gradient communication with backpropagation. During backpropagation, gradients are produced layer by layer in reverse order. Instead of waiting for all gradients to be computed before launching all-reduce, gradients are grouped into buckets. As soon as all gradients in a bucket become ready, an asynchronous all-reduce is launched for that bucket while the backward pass continues computing gradients for earlier layers. This overlap hides part of the communication latency behind computation and significantly improves training efficiency.

Let T_{comp} be total backward computation time, T_{comm} be total communication time and $T_{overlap}$ communication time hidden by overlap, then the effective iteration time becomes approximately:

$$T_{iter} \approx T_{comp} + T_{comm} - T_{overlap}$$

When overlap is highly effective, the iteration time approaches the larger of computation time and communication time rather than their sum. This bucket-based asynchronous communication strategy is one of the key techniques that makes PyTorch Distributed Data Parallel highly efficient, and I will implement this overlapping in the customized DP.

2.2 Related Parallel Strategy

This section briefly introduces other mainstream parallelization strategies commonly used in large-scale training systems, although this project only implements Data Parallelism (DP).

While standard DP replicates the full model on each GPU and incurs significant memory redundancy, the Zero Redundancy Optimizer (ZeRO) [2] introduces an optimization of DP and reduce its overhead by partitioning model states across devices. ZeRO Stage 1 shards optimizer states, Stage 2 additionally shards gradients, and Stage 3 further partitions model parameters, enabling much larger models to be trained under the same hardware constraints.

Beyond ZeRO, modern large-scale training often combines multiple forms of parallelism depending on the bottleneck being addressed. Pipeline Parallelism (PP) [3] splits the model across layers and executes different stages on different GPUs in a pipelining fashion. Tensor Parallelism (TP) [4] partitions weight matrix within layers across devices to accelerate computation-heavy operations. Sequence Parallelism (SP) [5] distributes activations along the sequence dimension to reduce memory usage, while Expert Parallelism (EP) [6], commonly used in Mixture-of-Experts models, assigns different expert modules to different GPUs and activates only a subset per input token. Together, these techniques form the core building blocks of contemporary distributed training systems for LLM.

3. Implementation

In this project, I implement Nano-DDP, including three progressively optimized versions of a simplified Data Parallelism based on PyTorch's collective communication primitives and *autograd* hook APIs. All versions inherit the same base class and share the synchronize procedure: gradients are communicated across GPUs using *all_reduce*(SUM) and then divided by *world_size* to obtain the averaged gradient. The three versions mainly differ in

how and when gradient synchronization is triggered and how communication is grouped. Overall, the design evolves from a simple “synchronous after-backward” baseline (V1), to fine-grained per-parameter communication (V2), and finally to a more efficient bucket-based aggregation strategy (V3). The V2 realized the overlapping between computation and communication, while the V3 strikes a better balance between communication overhead and delay, which more closely matches the behavior of original PyTorch DDP systems.

Nano-DDP V1: Post-backward Full Synchronization

In V1, gradient synchronization happens only after the entire backward pass is completed. Once all parameter gradients are ready, they are flattened into a single vector, and one *all_reduce*(SUM) operation is performed to aggregate gradients across all GPUs. A custom *register_multi_grad_hook* is used to trigger synchronization only when all gradients in the model are available.

Nano-DDP V2: Per-Parameter Overlapped Synchronization

V2 improves upon V1 by enabling communication to overlap with backward computation. Instead of waiting for the full backward pass, each parameter registers a *register_post_accumulate_grad_hook*, which triggers an *all_reduce* as soon as that parameter’s gradient becomes ready. This allows gradient synchronization to run asynchronously during backpropagation. A *register_multi_grad_hook* is still used as a final barrier to ensure correctness. Compared to V1, V2 significantly improves efficiency by hiding part of the communication latency behind computation.

Nano-DDP V3: Bucketed Gradient Synchronization

V3 further optimizes V2 by reducing the number of communication calls. Instead of performing one *all_reduce* per parameter, gradients are grouped into buckets based on a size threshold. Once a bucket is full, all gradients inside it are flattened and reduced together using a single asynchronous *all_reduce*. This bucket-based strategy significantly reduces communication overhead by decreasing the number of collective launches, while preserving overlap with backward computation. The implementation still relies on the same hook mechanism as V2, but achieves better scalability and performance, making it closer to the design of official PyTorch DDP.

4. Experiment

The experiments are conducted on a single-node 2 Nvidia 3090 GPU with NVLink. The task is GLUE MRPC [8] text classification using the 340M BERT-Large [7] with a maximum sequence length of 128. Optimization is performed using AdamW [9] with a learning rate of 2e-5, and

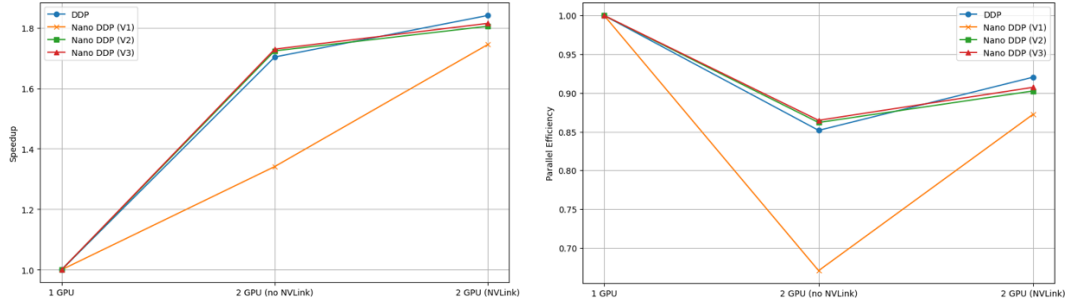


Figure 1: Speedup (left) and Parallel Efficiency (right) of Nano-DDP V1-V3 and PyTorch DDP.

data is sharded across GPUs using *DistributedSampler*. Each run is executed for 30 training steps, where the first 10 warmup steps are excluded from timing to ensure stable throughput measurement.

Based on this setup, I conduct experiments along three dimensions. First, I evaluate scalability of my implementations by varying the number of GPUs. Second, I compare the efficiency of Nano-DDP V1/V2/V3 against the official PyTorch DDP baseline, focusing on per-step execution time. This allows a direct assessment of how different synchronization strategies affect training performance. Finally, I use NVIDIA Nsight Systems to visualize execution traces and study the overlap between backward computation and gradient communication.

4.1 Scalability

In the experiments, I study scalability by varying the number of GPUs (up to 2 NVIDIA 3090 GPUs in total due to hardware limitations). To ensure a fair comparison across different GPU configurations, I keep the global batch size fixed, such that each GPU processes a local batch size equal to the global batch size divided by the number of GPUs (world size).

I evaluate scalability using two metrics: (1) the achieved speedup relative to single-GPU training (Figure 1 left) the parallel efficiency (Figure 1 right). The first metric measures the absolute wall-clock acceleration, while the second normalizes speedup by the number of devices.

The results show that with NVLink-enabled communication, the system achieves near-linear scalability, with speedup closely tracking the number of GPUs and correspondingly high parallel efficiency. This indicates that communication overhead is effectively reduced and well overlapped with computation.

In contrast, when NVLink is not available, scalability degrades noticeably. The speedup

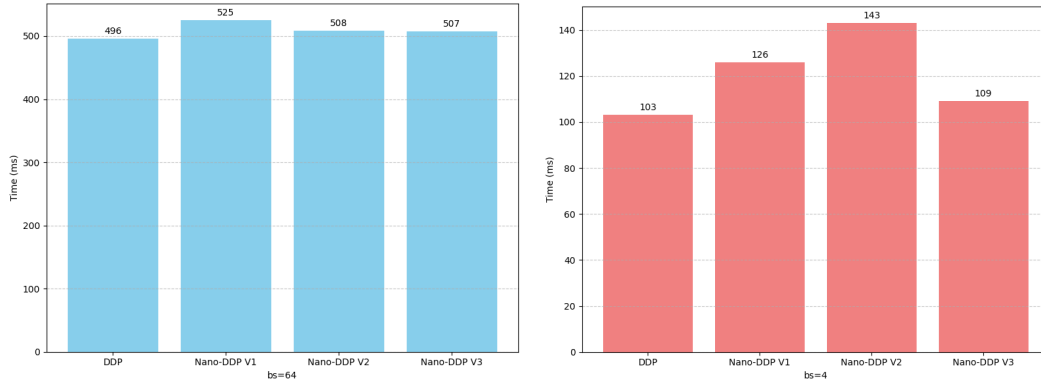


Figure 2: Execution time per step (ms) of Nano-DDP V1-V3 and PyTorch DDP with different batch size (left bs=64, right bs=4).

deviates from the ideal linear trend, leading to lower parallel efficiency. Among the evaluated methods, Nano-DDP v1 is affected the most, as it does not support computation–communication overlap, causing communication overhead to become the dominant bottleneck as the number of GPUs increases.

4.2 Efficiency

To evaluate the implementation efficiency of my implementation, I compare three versions of Nano-DDP against the official PyTorch DDP implementation. For each method, I measure the average wall-clock time per training step under the same experimental settings.

The results (Figure 2) are consistent with my expectations. Nano-DDP v1 exhibits the highest per-step latency, while Nano-DDP v3 achieves performance much closer to the official PyTorch DDP implementation. Among all methods, the official PyTorch DDP remains the fastest. This performance trend is expected: Nano-DDP v1 performs gradient synchronization only after the backward pass and therefore does not overlap communication with computation, whereas Nano-DDP v3 overlaps communication with backward computation as the official PyTorch DDP implementation.

The performance differences are more pronounced when the batch size is small. With smaller batch sizes, the amount of computation per step decreases while the communication cost remains largely unchanged, leading to a lower computation-to-communication ratio and making the benefits of computation–communication overlap more significant.

4.3 Kernel Profiling

Implementation	All CUDA Kernel Time	AllReduce Kernel Time	AllReduce Kernel Time%	AllReduce Kernel Launch
DDP	38461 ms	9423 ms	24.5%	2206
V1	30944 ms	2101 ms	6.8%	60
V2	39290 ms	10256 ms	26.1%	23580
V3	38891 ms	9688 ms	24.9%	2998

Table 1: Kernel statistics, including kernel time and number of launches.

To further verify the correctness of the implementation, I profiled each method using NVIDIA Nsight Systems. Specifically, I measured the total time spent in all-reduce operations, the number of all-reduce launches, and inspected the execution timelines.

The profiling results (Table 1) are consistent with the expected behavior. Although Nano-DDP v1 shows a smaller total all-reduce duration, all gradient synchronizations are launched only after the backward pass has completed, resulting in no overlap between communication and computation. Nano-DDP v2, Nano-DDP v3, and the official PyTorch DDP implementation exhibit similar total all-reduce time. Compared with Nano-DDP v2, however, both Nano-DDP v3 and PyTorch DDP use gradient bucketing, which significantly reduces the number of all-reduce launches and thereby lowers the kernel launch overhead.

The execution timelines (Appendix) provide further confirmation of this behavior. In the traces, the all-reduce kernels in Nano-DDP v1 are serialized with other kernels. In contrast, for Nano-DDP v2, Nano-DDP v3, and PyTorch DDP, all-reduce kernels are launched in parallel with backward computation, demonstrating effective overlap between communication and computation.

5. Summary

In this project, I implemented a simplified Distributed Data Parallel framework, Nano-DDP, with three progressively optimized versions: Nano-DDP v1 performs gradient synchronization only after the entire backward pass has completed; Nano-DDP v2 launches an all-reduce operation for each parameter as soon as its gradient is computed; and Nano-DDP v3 further introduces gradient bucketing to reduce the number of communication launches and improve overlap efficiency.

The experimental results show that these design choices have a substantial impact on performance. Nano-DDP v1 suffers from the highest communication overhead due to the

absence of communication–computation overlap, while Nano-DDP v2 improves efficiency by overlapping gradient synchronization with backward computation. By combining overlap with gradient bucketing, Nano-DDP v3 achieves performance close to the official PyTorch DDP implementation across both efficiency and scalability experiments.

Despite this strong performance, a gap remains between Nano-DDP v3 and PyTorch DDP. One important reason is that PyTorch DDP is implemented primarily in optimized C++/CUDA components tightly integrated with the *autograd* engine, allowing communication hooks and scheduling logic to be executed with minimal runtime overhead. In contrast, Nano-DDP registers hooks and orchestrates communication at the Python level, which introduces additional interpreter overhead and Python-to-C++ dispatch costs during training. Reducing this overhead by moving more of the implementation into lower-level C++ extensions or integrating more directly with the *autograd* engine is a promising direction for future work.

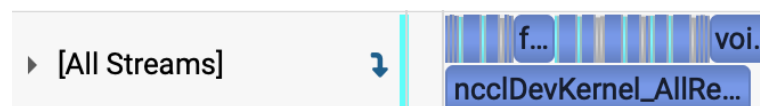
References

- [1] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., & Chintala, S. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. In Advances in Neural Information Processing Systems (NeurIPS), 2019.
- [2] Rajbhandari, S., Rasley, J., Ruwase, O., & Yuxiong He (2020). *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models*. In Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC20). IEEE/ACM.
- [3] Narayanan, D., Harlap, A., Phanishayee, A., Seshadri, V., Devanur, N., Ganger, G. R., & Matei Zaharia (2019). *PipeDream: Generalized Pipeline Parallelism for DNN Training*. In Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP), 1–15.
- [4] Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., & Bryan Catanzaro (2019). *Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism*. arXiv:1909.08053.
- [5] Korthikanti, V. A., Casper, J., Lym, S., et al. (2023). *Reducing Activation Recomputation in Large Transformer Models*. In Proceedings of Machine Learning and Systems (MLSys).
- [6] Lepikhin, D., Lee, H., Xu, Y., Chen, D., Firat, O., Huang, Y., Krikun, M., Shazeer, N., & Yonghui Wu (2021). *GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding*. In International Conference on Learning Representations (ICLR).

- [7] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL), 4171–4186.
- [8] Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., & Bowman, S. R. (2019). *GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding*. In International Conference on Learning Representations (ICLR), 2019.
- [9] Loshchilov, I., & Hutter, F. (2019). *Decoupled Weight Decay Regularization*. In International Conference on Learning Representations (ICLR), 2019.

Appendix

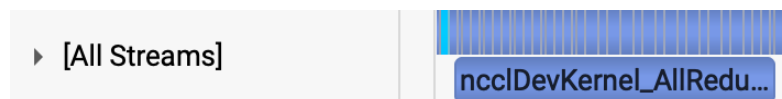
NVIDIA Nsight Systems Traces



PyTorch DDP



Nano-DDP V1



Nano-DDP V2



Nano-DDP V3

